

MiniHub Car

Contrato Técnico de Logs e Auditoria

Documento complementar à Arquitetura Técnica da Fase 2

Versão: 1.0

Escopo: Technical Logs, Correlation/Trace ID, Audit Log, AuditService, SQL PostgreSQL, segurança e retenção.

Princípio fundamental

Technical Logs explicam como o sistema se comportou. Audit Logs registram o que aconteceu, quem realizou ou provocou a operação, sobre qual recurso, em qual contexto e com qual resultado. Um mecanismo não substitui o outro.

1. Objetivo

Definir os contratos técnicos e o modelo de dados que permitirão ao MiniHub Car diagnosticar o sistema, correlacionar operações, rastrear ações de negócio e manter uma trilha de auditoria protegida e confiável.

A arquitetura preserva a separação já definida entre Technical Log, Audit Log, Application Event, Metric e Trace.

2. Separação dos mecanismos

Mecanismo	Finalidade	Exemplo
Technical Log	Diagnóstico técnico	Timeout do provedor de pagamento
Audit Log	Governança e rastreabilidade	Usuário alterou preço de uma oferta
Application Event	Comunicar fato do domínio	ORDER_PAID
Metric	Medir comportamento agregado	Latência média da API
Trace	Rastrear execução distribuída	Request → serviço → worker

3. Contrato técnico do Technical Log

Os logs devem ser estruturados, preferencialmente em JSON. Mensagens de texto livres não devem ser o mecanismo principal de logging.

```
{ "timestamp": "2026-09-07T18:30:15.245Z", "level": "INFO", "service": "orders", "environment": "production",  
  "event": "order_created", "operation": "createOrder", "trace_id": "uuid", "correlation_id": "uuid",  
  "request_id": "uuid", "actor_id": "uuid", "entity_type": "ORDER", "entity_id": "uuid", "duration_ms": 184,  
  "metadata": { "items_count": 3 } }
```

Campo	Tipo	Obrig.	Finalidade
timestamp	TIMESTAMPZ	Sim	Momento do log
level	ENUM/string	Sim	DEBUG, INFO, WARN, ERROR
service	VARCHAR	Sim	Módulo/serviço
environment	VARCHAR	Sim	Ambiente
event	VARCHAR	Sim	Evento técnico
operation	VARCHAR	Não	Operação executada
trace_id	UUID/string	Sim	Rastreamento
correlation_id	UUID/string	Sim	Correlação
request_id	UUID/string	Não	Requisição
actor_id	UUID	Não	Usuário/ator
entity_type	VARCHAR	Não	Recurso afetado
entity_id	UUID	Não	Identificador do recurso
duration_ms	INTEGER	Não	Duração
error_code	VARCHAR	Não	Código técnico
message	TEXT	Não	Mensagem legível
metadata	JSONB	Não	Dados técnicos adicionais

4. Níveis de log

DEBUG: diagnóstico detalhado. **INFO:** operação normal relevante. **WARN:** situação anormal sem falha total. **ERROR:** falha que impede ou compromete uma operação.

5. Dados que não devem ser registrados

Nunca registrar senha, password hash, tokens, refresh tokens, CVV, dados completos de cartão, secrets, API keys ou payloads contendo dados pessoais desnecessários. Payloads externos devem ser minimizados e sanitizados.

6. LoggerService e contexto de execução

Os módulos não devem utilizar `console.log` diretamente como mecanismo de produção. Deve existir uma abstração `LoggerService`, com `debug()`, `info()`, `warn()` e `error()`. O contexto de execução deve carregar `trace_id`, `correlation_id`, `request_id` e, quando aplicável, `actor_id`.

Request ■■■ trace_id ■■■ correlation_id ■■■ request_id ■■■ actor_id ↓ Application → Domain → Repository

7. Modelo de dados da auditoria

A tabela `AUDIT_LOG` é transversal. Não deverá possuir FK física obrigatória para todas as entidades auditadas; `entity_type` + `entity_id` permitem manter o mecanismo desacoplado.

Campo	Tipo PostgreSQL	Obrig.	Descrição
id	UUID	Sim	Identificador do registro
occurred_at	TIMESTAMPTZ	Sim	Data/hora da operação
actor_id	UUID	Não	Usuário responsável
actor_type	VARCHAR(30)	Sim	USER, ADMIN, SYSTEM, WORKER, WEBHOOK etc.
action	VARCHAR(100)	Sim	Ação de negócio
entity_type	VARCHAR(80)	Sim	Tipo do recurso
entity_id	UUID	Sim	Recurso afetado
old_state	VARCHAR(50)	Não	Estado anterior
new_state	VARCHAR(50)	Não	Estado posterior
result	VARCHAR(20)	Sim	SUCCESS, FAILURE, DENIED
source	VARCHAR(40)	Sim	API, WORKER, WEBHOOK etc.
ip_address	INET	Não	IP de origem
user_agent	TEXT	Não	Agente do cliente
correlation_id	UUID	Não	Correlação
trace_id	UUID	Não	Rastreamento técnico
reason	TEXT	Não	Justificativa
metadata	JSONB	Não	Informações adicionais

8. Instruções SQL — PostgreSQL

A seguir está uma implementação inicial compatível com PostgreSQL, alinhada ao modelo definido. UUIDs são utilizados como identificadores externos.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; CREATE TABLE audit_log ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
occurred_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP, actor_id UUID NULL, actor_type VARCHAR(30) NOT NULL,
action VARCHAR(100) NOT NULL, entity_type VARCHAR(80) NOT NULL, entity_id UUID NOT NULL, old_state VARCHAR(50)
NULL, new_state VARCHAR(50) NULL, result VARCHAR(20) NOT NULL, source VARCHAR(40) NOT NULL, ip_address INET
NULL, user_agent TEXT NULL, correlation_id UUID NULL, trace_id UUID NULL, reason TEXT NULL, metadata JSONB NULL,
CONSTRAINT ck_audit_result CHECK (result IN ('SUCCESS', 'FAILURE', 'DENIED')), CONSTRAINT ck_audit_actor_type
CHECK (actor_type IN ( 'USER', 'ADMIN', 'SYSTEM', 'WORKER', 'SCHEDULER', 'WEBHOOK', 'INTEGRATION' )) );
```

9. Índices recomendados

```
CREATE INDEX idx_audit_entity ON audit_log (entity_type, entity_id); CREATE INDEX idx_audit_actor ON audit_log
(actor_id); CREATE INDEX idx_audit_action ON audit_log (action); CREATE INDEX idx_audit_occurred_at ON audit_log
(occurred_at DESC); CREATE INDEX idx_audit_correlation ON audit_log (correlation_id); CREATE INDEX
idx_audit_trace ON audit_log (trace_id); CREATE INDEX idx_audit_entity_date ON audit_log (entity_type,
entity_id, occurred_at DESC);
```

10. Proteção contra UPDATE e DELETE

A aplicação não deve possuir fluxo normal de alteração ou exclusão de registros de auditoria. O controle efetivo deve ser aplicado também no banco por meio de privilégios.

```
-- Exemplo conceitual de usuário de aplicação: REVOKE UPDATE, DELETE ON audit_log FROM minihub_app; -- Permitir
somente inserção e consulta conforme o desenho -- de permissões do ambiente: GRANT INSERT, SELECT ON audit_log TO
minihub_app;
```

11. Exemplos de registros

```
-- Alteração de preço INSERT INTO audit_log ( actor_id, actor_type, action, entity_type, entity_id, old_state,
new_state, result, source, correlation_id, trace_id, reason, metadata ) VALUES (
'00000000-0000-0000-0000-000000000001', 'USER', 'OFFER_PRICE_CHANGED', 'OFFER',
'00000000-0000-0000-0000-000000000002', 'ACTIVE', 'ACTIVE', 'SUCCESS', 'API',
'00000000-0000-0000-0000-000000000003', '00000000-0000-0000-0000-000000000004', 'Atualização comercial',
'{"old_price": 89.90, "new_price": 99.90}::jsonb ); -- Tentativa de acesso indevido INSERT INTO audit_log (
actor_id, actor_type, action, entity_type, entity_id, result, source, reason, metadata ) VALUES (
'00000000-0000-0000-0000-000000000005', 'USER', 'OFFER_UPDATE', 'OFFER', '00000000-0000-0000-0000-000000000006',
'DENIED', 'API', 'OWNERSHIP_VIOLATION', '{"attempt": "update"}::jsonb );
```

12. Auditoria transacional

Para operações críticas, a alteração de negócio, o registro de auditoria e o Outbox devem participar da mesma transação. Se a operação sofrer rollback, a auditoria de uma mudança que não ocorreu não deve permanecer como SUCCESS.

```
BEGIN; -- 1. alteração de negócio -- UPDATE / INSERT / comando de domínio -- 2. registro da auditoria INSERT INTO
audit_log (...); -- 3. registro do evento na OUTBOX INSERT INTO outbox (...); COMMIT;
```

Tentativas DENIED podem ser auditadas mesmo sem alteração da entidade, pois representam eventos relevantes de segurança.

13. Catálogo inicial de ações auditáveis

USER_CREATED, USER_SUSPENDED, SELLER_REQUESTED, SELLER_APPROVED, SELLER_SUSPENDED, OFFER_CREATED, OFFER_ACTIVATED, OFFER_PAUSED, OFFER_PRICE_CHANGED, STOCK_REGISTERED, STOCK_RESERVED, STOCK_COMMITTED, STOCK_ADJUSTED, ORDER_CREATED, ORDER_CANCELLED, PAYMENT_CREATED, PAYMENT_APPROVED, PAYMENT_FAILED, REFUND_REQUESTED, REFUND_COMPLETED, FULFILLMENT_CONFIRMED, SHIPMENT_CREATED, SHIPMENT_DISPATCHED, GARAGE_ITEM_CREATED, DISPATCH_REQUEST_CREATED, PRE_SALE_CREATED, PRE_SALE_RESERVATION_CREATED, PRE_SALE_RESERVATION_CANCELLED, RAFFLE_CREATED, RAFFLE_DRAWN, POST_PUBLISHED, POST_HIDDEN e POST_REMOVED.

14. AuditService

A aplicação deve utilizar uma abstração `AuditService` para registrar ações, evitando SQL direto da camada de negócio.

```
audit.record({ action: "OFFER_PRICE_CHANGED", entityType: "OFFER", entityId: offer.id, actorId: user.id,
actorType: "USER", oldState: "ACTIVE", newState: "ACTIVE", result: "SUCCESS", source: "API", reason:
"Atualização comercial", metadata: { oldPrice: 89.90, newPrice: 99.90 } }));
```

15. Consultas SQL úteis

```
-- Histórico de uma entidade SELECT * FROM audit_log WHERE entity_type = 'OFFER' AND entity_id = :offer_id ORDER
BY occurred_at DESC; -- Ações realizadas por um usuário SELECT occurred_at, action, entity_type, entity_id,
result FROM audit_log WHERE actor_id = :actor_id ORDER BY occurred_at DESC; -- Tentativas negadas SELECT
occurred_at, actor_id, action, entity_type, entity_id, reason FROM audit_log WHERE result = 'DENIED' ORDER BY
occurred_at DESC; -- Reconstruir uma alteração de estado SELECT occurred_at, action, old_state, new_state,
actor_id, result FROM audit_log WHERE entity_type = 'ORDER' AND entity_id = :order_id ORDER BY occurred_at ASC;
```

16. Retenção e crescimento

Technical Logs podem ter retenção mais curta conforme ambiente e necessidade operacional. Audit Logs devem possuir retenção definida de acordo com requisitos legais, operacionais e de governança. Não fixar um prazo definitivo sem validação jurídica. O particionamento por `occurred_at` deve ser considerado quando o volume justificar.

17. Segurança da auditoria

A consulta à auditoria deve ser restrita a perfis autorizados. O acesso à própria auditoria deve ser auditável. Ownership e autorização devem ser validados no backend. Dados pessoais devem ser minimizados. Auditoria não deve expor segredos ou dados de pagamento.

18. Critérios de aceite técnicos

Given que uma operação crítica altera uma entidade, When a transação for confirmada, Then a alteração e seu Audit Log deverão ser confirmados de forma consistente.

Given que uma operação é negada por autorização ou ownership, When a tentativa ocorrer, Then deverá existir registro de auditoria com `result=DENIED` quando a regra de segurança definir a tentativa como auditável.

Given que um usuário consulta o histórico de uma entidade, When possuir permissão adequada, Then a API deverá retornar os registros ordenados por `occurred_at`.

Given que um registro de auditoria existe, When uma rotina comum tentar alterá-lo ou excluí-lo, Then a operação deverá ser impedida.

Given que um log é produzido, When contiver dados sensíveis, Then a camada de logging deverá sanitizar ou omitir esses dados.

19. Checklist de implementação

- Contrato JSON do Technical Log definido
- `LoggerService` implementado
- Execution Context implementado
- Correlation ID propagado
- Trace ID integrado
- Sanitização de dados sensíveis
- `AUDIT_LOG` criado via migration
- `AuditService` implementado
- Catálogo de ações auditáveis definido
- Auditoria de mudanças de estado
- Auditoria de operações `DENIED`

- Auditoria integrada às transações críticas
- UPDATE/DELETE protegidos
- Índices criados
- Permissões de consulta definidas
- Testes de auditoria implementados
- Monitoramento e retenção definidos

20. Regra final

Toda operação relevante do MiniHub Car deverá possuir contexto suficiente para ser diagnosticada, correlacionada e monitorada, mantendo separação entre logs técnicos, auditoria, eventos, métricas e rastreamento de execução, sem registrar dados sensíveis desnecessários.

A auditoria deverá ser tratada como evidência operacional: imutável, protegida, rastreável, orientada a ações de negócio e consistente com as transações que registra.